SESSION 6 REFERENCE GUIDE: SLICING, GROUPING & MULTI-METRIC AGGREGATION

This guide provides a reference for `.groupby()`, `.agg()`, `.sort_values()`, and `.value_counts()` syntax in Pandas introduced in Session 6.

1. The GroupBy Pattern (`.groupby()`)

Grouping splits data into discrete subsets based on a categorical column.

```python

import pandas as pd

# Group by a single category and sum a specific column
category_totals = df.groupby("product_category")["revenue"].sum()

# Group by a single category and calculate multiple column sums
category_multi = df.groupby("product_category")[["revenue",
"units_sold"]].sum()
```

2. Multi-Metric Aggregation (`.agg()`)

Apply different statistical functions to different columns simultaneously using a dictionary syntax.

```python

summary_df = df.groupby("product_category").agg({
    "revenue": ["sum", "mean"],      # Total revenue and average order
value
    "units_sold": "sum",             # Total unit volume
    "order_id": "count"              # Transaction count
})
```

3. Sorting & Ranking (`sort_values()`)

Isolate top and bottom performers by sorting aggregated outputs.

```python

# Sort aggregated DataFrame in descending order by total revenue
sorted_df = summary_df.sort_values(by=("revenue", "sum"), ascending=False)

# Sort single-level aggregated DataFrame
sorted_single = category_totals.sort_values(ascending=False)
```

4. Frequency Distribution (`value_counts()`)

Count occurrences of unique values within a categorical column (Series).

```python
# Count frequency of orders by customer tier
tier_counts = df["customer_tier"].value_counts()

# Return proportions instead of raw counts
tier_proportions = df["customer_tier"].value_counts(normalize=True)
```

5. The Analytical Aggregation Workflow

```
[Clean Dataset] --> [Groupby Category] --> [Apply .agg() Metrics] --> [Sort Values] --> [Interpret Business Drivers]
```